

Reactivity & Lifecycle

INFO

Lesinhoud

Je leert wat reactiviteit is in Vue.js, hoe je reactieve data maakt met `ref` en `reactive`, en hoe je deze data bindt aan de template. Je leert ook over lifecycle hooks en hoe je deze gebruikt om code uit te voeren op specifieke momenten in de levenscyclus van een component.

Doelstellingen

De student(e) kan:

- Werken met een Command Line Interface (CLI) / Node Package Manager (NPM) voor het installeren, updaten en verwijderen van pakketten en modules binnen een project.
- Een single-page frontend applicatie ontwikkelen met Vue.js.
- Zelfstandig de bovenstaande technologieën combineren tot een werkende webapplicatie.

▼ Inhoud

- [Interpolatie](#)
- [Wat is Reactivity?](#)
 - [ref in detail](#)
- [One way vs Two way binding](#)
 - [One way binding](#)
 - [Two way binding](#)

- [ref vs reactive](#)
 - [Destructuring](#)
- [Lifecycle Hooks](#)
- [Watch en Computed](#)
- [Directives](#)
 - [v-bind](#)
 - [v-on](#)
 - [v-if, v-else](#)
 - [v-show](#)
 - [v-for](#)
 - [v-model](#)
- [Meer leren](#)

Interpolatie

In Vue.js kan je data **binden** (Nederlands: koppelen) aan de template via **interpolatie**. Dit betekent dat je de waarde van een variabele of expressie in de template kan weergeven door deze tussen dubbele accolades (`{{ }}`) te plaatsen.

```
<script setup>
const message = "Hello, Vue.js!";
const number = 42;
</script>

<template>
  <p>{{ message }}</p>
  <p>The answer to life, the universe and everything is
    {{ number }}.</p>
```

vue

```
<p>2 + 2 = {{ 2 + 2 }}</p>
</template>
```

Door `{{ }}` te gebruiken, geef je de waarden van `message`, `number` en zelfs expressies zoals `2 + 2` direct in de HTML weer.

Wat is Reactivity?

Reactivity (Nederlands: reactiviteit) is een kernconcept in moderne frontend frameworks (Vue.js, React, Angular, ...). Het verwijst naar de applicatie die automatisch reageert op veranderingen in data en de gebruikersinterface (UI) bijwerkt zonder dat de developer handmatig de DOM moet manipuleren.

In Vue.js is reactivity gebaseerd op een systeem waarbij je via `composables` (Composition API) data reactief maakt. Je doet dit via de composables `ref` en `reactive`. `Composable` is een ander woord voor "herbruikbare functie die specifieke functionaliteit biedt binnen de Vue Composition API".

Neem onderstaande code over in een `view`-component:

```
<script setup>
let count = 0;

function increment() {
  count++;
  console.log(count);
}
</script>

<template>
<div>
  <p>Count: {{ count }}</p>
```

vue

```
<button @click="increment">Increment</button>
</div>
</template>
```

Oefening

Klik op `Increment` en observeer wat er gebeurt met de `count` -waarde op de pagina. Observeer ook wat er in de console wordt gelogd. Wat valt hier op?

▼ Antwoord

De `count` -waarde in de console wordt verhoogd bij elke klik, maar de waarde op de pagina blijft altijd `0`.

Vue werkt de geïnterpoleerde waarde `{{ count }}` in de template niet bij wanneer de `count` -variabele verandert. Dit komt omdat Vue niet weet dat je de waarde van `count` hebt gewijzigd. Vue kan alleen wijzigingen in de UI bijhouden als je de data reactief maakt.

Om te zorgen dat Vue weet dat `count` een reactieve variabele is, moet je deze definiëren met de `ref` -composable:

```
<script setup>
import { ref } from "vue";

const count = ref(0);

function increment() {
  count.value++;
  console.log(count.value);
}
</script>

<template>
<div>
```

vue

```
<p>Count: {{ count }}</p>
<button @click="increment">Increment</button>
</div>
</template>
```

Oefening

Klik opnieuw op `Increment` en observeer wat er gebeurt met de `count` -waarde op de pagina. Observeer ook wat er in de console wordt gelogd. Wat valt hier op?

▼ Antwoord

De `count` -waarde op de pagina wordt nu ook verhoogd bij elke klik, net zoals in de console. Dit komt omdat Vue nu weet dat `count` een reactieve variabele is dankzij het gebruik van `ref`.

Merk op dat je de `ref` -composable importeert vanuit de `vue` -module. Dit is een standaard composable die Vue aanbiedt om primitieve waarden (zoals nummers, strings, booleans) reactief te maken. Merk op dat `count` nu een `const` is i.p.v. een `let`. Dit komt omdat `ref` een object teruggeeft waarvan je de waarde (`value`) kan aanpassen. Merk op dat je de waarde van `count` via `count.value` wijzigt en niet rechtstreeks via `count`. Merk op dat je in de template niet `{{ count.value }}` gebruikt, maar gewoon `{{ count }}`. Vue zorgt er automatisch voor dat de `.value` wordt opgehaald wanneer je een `ref` in de template gebruikt.

ref in detail

De `ref` -composable wordt gebruikt om primitieve waarden (zoals nummers, strings, booleans) reactief te maken.

Om beter te begrijpen hoe `ref` werkt, zal er een vereenvoudigde implementatie van een `ref`-functie gegeven worden:

```
function ref(initialValue) {  
  let value = initialValue;  
  
  // Hier wordt nog logica toegevoegd om de waarde in de  
  // gaten te houden  
  
  return {  
    value: value,  
  };  
}
```

Wanneer je deze `ref`-functie gebruikt om een reactieve variabele te maken:

```
const count = ref(0); // count is nu een object met een  
// 'value'-eigenschap waarvan de waarde 0 is.  
console.log(count); // { value: 0 }  
count.value++;  
console.log(count); // { value: 1 }
```

Hier valt op dat je een object terugkrijgt met een `value`-eigenschap. Dit is hoe Vue de waarde van een `ref` bijhoudt.

Wanneer je `count.value++` uitvoert, wijzig je de `value`-eigenschap van het object. Vue detecteert deze wijziging en werkt de UI bij.

One way vs Two way binding

In Vue.js werkt data binding standaard `one way`. Dit betekent dat de data van de component naar de template stroomt, maar niet andersom.

Wanneer je een variabele wijzigt in de template (bijvoorbeeld via een input-veld), werkt je de variabele in de component niet automatisch bij.

One way binding

```
<script setup>
import { ref } from "vue";

const message = ref("Hello, Vue.js!");
</script>

<template>
  <input type="text" :value="message" />
  <p>{{ message }}</p>
</template>
```

In dit voorbeeld bind je de `message` -variabele aan de `value` -eigenschap van het input-veld via `:value="message"`. Dit is een **one way binding**: wanneer je de waarde van `message` in de component wijzigt, werkt je het input-veld bij. Maar wanneer je iets in het input-veld typt, werkt je de `message` -variabele in de component niet automatisch bij.

Je kan dit oplossen door naar het `input` -event van het input-veld te luisteren en de `message` -variabele bij te werken:

```
<script setup>
import { ref } from "vue";

const message = ref("Hello, Vue.js!");

function updateMessage(event) {
  message.value = event.target.value;
}
</script>
```

```

<template>
  <input type="text" :value="message"
  @input="updateMessage" />
  <p>{{ message }}</p>
</template>

```

Dit patroon komt vaak voor, dus Vue biedt je een eenvoudigere manier om dit te doen via **two way binding** met de **v-model** -directive.

Two way binding

Je gebruikt onderstaande code met **v-model** om een **two way binding** te creëren tussen de **message** -variabele en het input-veld. Dit is eenvoudiger en overzichtelijker dan het voorbeeld met **:value** en **@input**.

```

<script setup>
  import { ref } from "vue";

  const message = ref("Hello, Vue.js!");
</script>

<template>
  <input type="text" v-model="message" />
  <p>{{ message }}</p>
</template>

```

vue

Je kan two way binding dus op twee manieren realiseren:

- Handmatig door een reactieve variabele te binden aan een **prop** van een HTML-element of component en naar een **emit** -event te luisteren om de variabele bij te werken.
- Automatisch door de **v-model** -directive te gebruiken die dit achterliggend voor je regelt.

Oefening

Maak een formulier met drie tekstvelden: `voornaam`, `achternaam` en `e-mailadres`. Gebruik `v-model` om de waarden van deze velden te binden aan reactieve variabelen. Toon de ingevulde waarden onder het formulier in een `ol`-lijst met drie `li`-items.

ref vs reactive

De `ref`-composable wordt gebruikt om primitieve waarden (zoals nummers, strings, booleans) reactief te maken. Voor complexere data zoals objecten en arrays wordt de `reactive`-composable gebruikt.

```
<script setup>
import { ref } from "vue";

const firstName = ref("John");
const lastName = ref("Duck");
const email = ref("john@duck.com");
</script>

<template>
  <form>
    <input type="text" v-model="firstName"
placeholder="Voornaam" />
    <input type="text" v-model="lastName"
placeholder="Achternaam" />
    <input type="email" v-model="email" placeholder="E-
mailadres" />
  </form>

  <ol>
    <li>Voornaam: {{ firstName }}</li>
    <li>Achternaam: {{ lastName }}</li>
    <li>E-mailadres: {{ email }}</li>
```

vue

```
</ol>
</template>
```

Dit werkt, maar dit kan ook gedaan worden met één enkel object dat alle drie de velden bevat. Hiervoor wordt `reactive` gebruikt:

```
vue
<script setup>
import { reactive } from "vue";

const form = reactive({
  firstName: "John",
  lastName: "Duck",
  email: "john@duck.com",
});
</script>

<template>
  <form>
    <input type="text" v-model="form.firstName"
placeholder="Voornaam" />
    <input type="text" v-model="form.lastName"
placeholder="Achternaam" />
    <input type="email" v-model="form.email"
placeholder="E-mailadres" />
  </form>

  <ol>
    <li>Voornaam: {{ form.firstName }}</li>
    <li>Achternaam: {{ form.lastName }}</li>
    <li>E-mailadres: {{ form.email }}</li>
  </ol>
</template>
```

Het gebruik van `ref` of `reactive` hangt af van het type data dat gebruikt wordt:

- Gebruik `ref` voor primitieve waarden zoals nummers, strings en booleans.
- Gebruik `reactive` voor objecten en arrays.

Wanneer reactieve data gebruiken?

Wanneer data in de template getoond wordt en dit moet altijd up-to-date zijn, dan is er nood aan reactieve data (`ref` of `reactive`). Wanneer data één waarde heeft en niet zal veranderen, is het oké om gewone niet-reactieve variabelen te gebruiken.

Destructuring

Wanneer in JavaScript eigenschappen van een object toegewezen worden aan losse variabelen, zoals hieronder:

```
js
const person = {
  firstName: "John",
  lastName: "Duck",
};

const firstName = person.firstName;
const lastName = person.lastName;

console.log(firstName); // "John"
console.log(lastName); // "Duck"
```

Dan is er een kortere syntax om dit te doen, genaamd `destructuring` :

```
js
const person = {
  firstName: "John",
  lastName: "Duck",
};

const { firstName, lastName } = person;
```

```
console.log(firstName); // "John"  
console.log(lastName); // "Duck"
```

Let op met destructuring van reactive objects

Wanneer je **destructuring** toepast op een **reactive** -object, gaat de reactiviteit verloren voor de losse variabelen. Dit komt omdat de losse variabelen niet langer verwijzen naar het originele **reactive** -object.

```
import { reactive } from "vue";  
  
const person = reactive({  
  firstName: "John",  
  lastName: "Duck",  
});  
  
const { firstName, lastName } = person; // firstName en  
lastName zijn nu gewone niet-reactieve variabelen  
  
console.log(firstName); // "John"  
console.log(lastName); // "Duck"
```

js

Om dit te vermijden, kan je de **toRefs** -composable gebruiken om de eigenschappen van een **reactive** -object om te zetten naar **ref** -objecten:

```
import { reactive, toRefs } from "vue";  
  
const person = reactive({  
  firstName: "John",  
  lastName: "Duck",  
});  
  
const { firstName, lastName } = toRefs(person); //
```

js

firstName en lastName zijn nu ref-objecten

```
console.log(firstName.value); // "John"  
console.log(lastName.value); // "Duck"
```

Dit is een eenvoudige manier om het onderstaande te bereiken:

```
import { reactive, ref } from "vue";  
  
const person = reactive({  
  firstName: "John",  
  lastName: "Duck",  
});  
  
const firstName = ref(person.firstName);  
const lastName = ref(person.lastName);  
  
console.log(firstName.value); // "John"  
console.log(lastName.value); // "Duck"
```

js

Lifecycle Hooks

Lifecycle hooks zijn speciale functies die Vue aanroept op specifieke momenten in de levenscyclus van een Vue-component. Deze hooks stellen je jezelf in staat om code uit te voeren wanneer een component wordt gemaakt, bijgewerkt of vernietigd.

Hier zijn enkele van de meest gebruikte lifecycle hooks in Vue.js:

- **onMounted** : Vue roept dit aan nadat de component is gemount (in de DOM is geplaatst). Dit is een goed moment om API-calls te doen of DOM-gerelateerde logica uit te voeren.
- **onUpdated** : Vue roept dit aan nadat de component is bijgewerkt (bijvoorbeeld na een verandering in de data). Dit is handig voor het

uitvoeren van logica die afhankelijk is van de bijgewerkte DOM.

- **onUnmounted** : Vue roept dit aan voordat de component wordt vernietigd. Dit is een goed moment om code op te ruimen, zoals het verwijderen van event listeners of het clearen van timers.

Van deze drie hooks is **onMounted** de meest gebruikte omdat het vaak nodig is om data op te halen die de component nodig heeft om de data weer te geven.

Voeg onderstaande code toe in **src/components/LifecycleDemo.vue** :

```
vue

<script setup>
import { ref, onMounted, onUpdated, onUnmounted } from
"vue";

const count = ref(0);

let intervalId = null;

onMounted(() => {
  console.log("Component is gemount");
  intervalId = setInterval(() => {
    count.value++;
  }, 1000);
});

onUpdated(() => {
  console.log("Component is bijgewerkt. Huidige count:",
count.value);
});

onUnmounted(() => {
  console.log("Component wordt vernietigd");
  clearInterval(intervalId);
});
</script>

<template>
```

```
<div>
  <p>Count: {{ count }}</p>
</div>
</template>
```

Gebruik deze component in een view, bijvoorbeeld in

`src/views/PageOne.vue` :

```
<script setup>
import LifecycleDemo from
"@/components/LifecycleDemo.vue";

const showComponent = ref(true);
</script>

<template>
  <h1>Page One</h1>

  <LifecycleDemo v-if="showComponent" />
  <button @click="showComponent = !showComponent">
    Toggle LifecycleDemo Component
  </button>
</template>
```

vue

Navigeer naar deze pagina in de browser. Denk eraan dat de view moet toegevoegd zijn aan de router.

Oefening

Bekijk de console-output bij het uitvoeren van bovenstaande code.

Verplaats de lifecycle hooks van plaats door ze in een andere volgorde te zetten. Wat valt op?

▼ Antwoord

De volgorde van de lifecycle hooks in de code maakt niet uit. Vue.js zorgt ervoor dat ze op de juiste momenten worden aangeroepen, ongeacht hun volgorde in de code.

Oefening

Wanneer wordt `onUnmounted` aangeroepen?

▼ Antwoord

`onUnmounted` wordt aangeroepen wanneer de component uit de DOM wordt verwijderd, in dit geval wanneer de `v-if`-directive evalueert naar `false`.

Oefening

Wat is het risico als de `clearInterval`-functie niet aangeroepen wordt in de `onUnmounted`-hook?

▼ Antwoord

Wanneer de `clearInterval` niet aangeroepen wordt, dan blijft de interval timer actief, zelfs nadat de component is vernietigd. Dit kan leiden tot `memory leaks` en verbruikt onnodig rekenkracht van de computer.

Watch en Computed

In Vue.js zijn `watch` en `computed` twee mechanismen waarmee je reageert op veranderingen in data, maar je gebruikt ze op verschillende manieren.

- **computed** properties zijn afgeleide waarden die Vue automatisch bijwerkt wanneer de onderliggende reactieve data verandert. Je gebruikt ze voor het berekenen van waarden op basis van andere waarden.
- **watch** gebruik je om specifieke reactieve data te observeren en een functie uit te voeren wanneer die data verandert. Het is handig voor het uitvoeren van side effects, zoals het maken van API-calls of het bijwerken van andere niet-reactieve waarden.

```

<script setup>
import { ref, computed, watch } from "vue";

const count = ref(0);
const doubleCount = computed(() => count.value * 2);

watch(count, (newValue, oldValue) => {
  console.log(`Count changed from ${oldValue} to ${newValue}`);
});
</script>

<template>
  <div>
    <p>Count: {{ count }}</p>
    <p>Double Count: {{ doubleCount }}</p>
    <button @click="count++">Increment</button>
  </div>
</template>

```

In dit voorbeeld:

- **doubleCount** is een **computed property** die Vue automatisch bijwerkt wanneer **count** verandert.
- De **watch** -functie observeert veranderingen in **count** en logt de oude en nieuwe waarde naar de console telkens wanneer je **count** bijwerkt.

Meestal is `computed` voldoende, je gebruikt `watch` alleen wanneer `computed` niet volstaat, bijvoorbeeld wanneer je een `side effect` moet uitvoeren bij een verandering in data (bijvoorbeeld, wanneer de `count` 10 bereikt moet je een bericht tonen).

```
vue

<script setup>
import { ref, computed, watch } from "vue";

const count = ref(0);
const doubleCount = computed(() => count.value * 2);
const message = ref("");

watch(count, (newValue) => {
  if (newValue === 10) {
    message.value = "Count heeft de waarde 10 bereikt!";
  } else {
    message.value = "";
  }
});
</script>

<template>
  <div>
    <p>Count: {{ count }}</p>
    <p>Double Count: {{ doubleCount }}</p>
    <button @click="count++">Increment</button>
    <p v-if="message">{{ message }}</p>
  </div>
</template>
```

Directives

Je gebruikt reactieve variabelen vaak in combinatie met `directives` om de data te binden aan HTML-elementen of componenten.

Directives zijn speciale attributen in Vue.js die beginnen met `v-`. Ze bieden je extra functionaliteit aan HTML-elementen en componenten. Enkele van de meest gebruikte directives zijn:

- `v-bind` (verkorte syntax: `:`): Gebruik je om attributen van HTML-elementen of componenten te binden aan data in de Vue-component.
- `v-on` (verkorte syntax: `@`): Gebruik je om event listeners toe te voegen aan HTML-elementen of componenten.
- `v-if`: Gebruik je om elementen conditioneel weer te geven op basis van een expressie.
 - `v-else`: Gebruik je in combinatie met `v-if` om een alternatief element weer te geven wanneer de `v-if`-voorwaarde niet waar is.
- `v-show`: Gebruik je om de zichtbaarheid van een element te regelen via CSS (`display: none;`), zonder het element uit de DOM te verwijderen.
- `v-for`: Gebruik je om een lijst van elementen te renderen.
- `v-model`: Gebruik je voor two-way data binding tussen een formulier element en een variabele in de Vue-component.

v-bind

Je gebruikt de `v-bind`-directive om attributen van HTML-elementen of componenten te binden aan data in de Vue-component. Dit maakt het mogelijk om dynamische waarden toe te wijzen aan attributen.

```
<script setup>
import { ref } from "vue";
const url = ref("https://9gag.com/gag/a9W9WyL");
</script>

<template>
  <a v-bind:href=url" target="_blank">Post bekijken</a>
  <a :href=url" target="_blank">Post bekijken</a>
</template>
```

vue

Conventie

In dit opleidingsonderdeel wordt de verkorte syntax van `v-bind` (dus met `:`) gebruikt.

Wel doen - verkorte syntax

```
<a :href="url" target="_blank">Post bekijken</a>
```

html

Niet doen - volledige syntax

```
<a v-bind:href="url" target="_blank">Post bekijken</a>
```

html

v-on

Je gebruikt de `v-on` -directive om event listeners toe te voegen aan HTML-elementen of componenten. Dit maakt het mogelijk om te reageren op gebruikersinteracties zoals klikken, typen, etc.

```
<script setup>
import { ref } from "vue";
const count = ref(0);

function increment() {
  count.value++;
}
</script>

<template>
  <div>
    <p>Count: {{ count }}</p>
    <button v-on:click="increment">Increment</button>
    <button @click="increment">Increment</button>
  </div>
</template>
```

vue

```
</div>
</template>
```

Conventie

In dit opleidingsonderdeel wordt de verkorte syntax van `v-on` (dus met `@`) gebruikt.

Wel doen - verkorte syntax

```
<button @click="increment">Increment</button>
```

html

Niet doen - volledige syntax

```
<button v-on:click="increment">Increment</button>
```

html

v-if, v-else

Je gebruikt de `v-if` -directive om elementen conditioneel weer te geven op basis van een expressie. Als de expressie `true` is, geef je het element weer, anders rendert je het element niet in de DOM.

```
<script setup>
import { ref } from "vue";
const isVisible = ref(true);
</script>

<template>
  <div>
    <button @click="isVisible = !isVisible">Toggle
    Visibility</button>

    <div v-if="isVisible">This content is visible.</div>
```

vue

```
<div v-else>This content is hidden.</div>

<div v-if="!isVisible">This content is hidden.</div>
</div>
</template>
```

v-show

Je gebruikt de `v-show` -directive om de zichtbaarheid van een element te regelen via CSS (`display: none;`), zonder het element uit de DOM te verwijderen. Dit is handig wanneer je de zichtbaarheid van een element vaak moet togglen zonder dat het telkens opnieuw gerenderd wordt.

```
<script setup>
import { ref } from "vue";
const isVisible = ref(true);
</script>

<template>
  <div>
    <button @click="isVisible = !isVisible">Toggle
    Visibility</button>

    <div v-show="isVisible">This content is visible.
  </div>
    <div v-show="!isVisible">This content is hidden.
  </div>
  </div>
</template>
```

vue

v-for

Je gebruikt de `v-for` -directive om een lijst van elementen te renderen op basis van een array of object. Dit maakt het mogelijk om dynamisch meerdere elementen te genereren.

```
<script setup>
import { ref } from "vue";
const items = ref(["Kwik", "Kwek", "Kwak"]);
</script>

<template>
  <ul>
    <li v-for="(item, index) in items" :key="index">{{
item }}</li>
  </ul>
</template>
```

Belangrijk - `:key` -attribuut

Bij het gebruik van `v-for` is het belangrijk om altijd een uniek `:key` -attribuut toe te voegen aan elk item in de lijst. Dit helpt Vue om efficiënt te bepalen welke items zijn gewijzigd, toegevoegd of verwijderd, wat de prestaties verbetert en ervoor zorgt dat de componenten correct worden bijgewerkt zonder onnodig opnieuw te renderen.

Bij meerdere lijsten op een pagina is het belangrijk dat de `:key` -waarden uniek zijn over alle lijsten heen. In bovenstaand voorbeeld zou dat bijvoorbeeld kunnen door `:key="'item-' + index"` te gebruiken in plaats van alleen `:key="index"`. Een tweede lijst zou dan bijvoorbeeld `:key="'other-item-' + index"` kunnen gebruiken.

v-model

Je gebruikt de `v-model` -directive voor two-way data binding tussen een formulier element en een variabele in de Vue-component. Dit maakt het eenvoudig om de waarde van een invoerveld te koppelen aan een reactieve variabele.

Zonder `v-model` :

vue

```
<script setup>
import { ref } from "vue";
const inputValue = ref("");

function updateValue(event) {
  inputValue.value = event.target.value;
}
</script>

<template>
  <input :value="inputValue" @input="updateValue" />
</template>
```

Dit is een patroon dat vaak voorkomt, je koppelt het `value`-attribuut aan een variabele en je luistert naar het `input`-event om de variabele bij te werken wanneer je de inhoud van het invoerveld verandert. Omdat dit patroon zo vaak voorkomt, biedt Vue je de `v-model`-directive aan om dit eenvoudiger te maken:

vue

```
<script setup>
import { ref } from "vue";
const inputValue = ref("");
</script>

<template>
  <input v-model="inputValue" />
</template>
```

Dit doet exact hetzelfde als het vorige voorbeeld, maar dan met minder code.

Meer leren

Bekijk de officiële Vue.js-documentatie voor meer informatie over [Reactivity](#) en [Lifecycle Hooks](#).

Previous page
[Oefeningen](#)

Next page
[Oefeningen](#)